
Netflix GenRec: Toward LLM-native Recommendation

GenRec: An LLM-Backed Recommendation Ranker at Netflix, Aug 2026

Traditional Recommender Stack

Raw Signals

User



age gender location
device tenure ...

Item



category brand price
attributes tags ...

Interaction



click view purchase
add_to_cart like ...

Hand-Crafted Features

User features

0.12 0.47 0.03 0.86 0.21 ...

Item features

0.33 0.18 0.76 0.42 0.09 ...

Interaction features

0.61 0.27 0.12 0.54 0.37 ...

Interaction crosses

U×I U×T I×T U×I×T
U² I² T² ...

Thousands of engineered signals

Customized Models

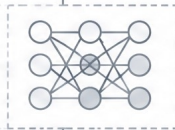
User tower



Item tower



Feature
interaction
network



CTR

CVR

Rating

Other

Bespoke architectures

New Content Type



Text
Image
Video
Audio
Code
Other

High
effort

High engineering friction

User, item, and interaction
features

Higher-order feature
interactions

Two-tower, DLRM, and
multi-task models

New content requires pipeline
and infrastructure changes

The LLM Paradigm Shift

Traditional RecSys



Hand-crafted features



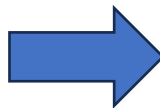
Bespoke model architectures



Feature engineering



Classic RecSys infrastructure



LLM-based recommender



Verbalized context



Shared foundation backbone



Context engineering



LLM serving infrastructure

Motivation for the LLM paradigm shift

- **Variety of specialized machine learning** models are being developed, each catering to distinct needs including “Continue Watching” and “Today’s Top Picks for You.”
- **Maintenance** of the recommender system became quite costly.
- It was difficult to **transfer innovations** from one model to another, given that most are independently trained despite using common data sources.
- A new recommender system architecture where member preference learning is centralized, enhancing **accessibility and utility across different use cases**.

Why an out-of-the-box LLM is not a ranker

Popularity Bias

Base models tend to over-recommend globally popular content, ignoring the nuanced, long-tail preferences of individual members.

Hallucination

Without strict constraints, generative models frequently hallucinate out-of-catalog titles that cannot be played.

Weak Personalization

Off-the-shelf models lack deep understanding of specific user interaction histories and domain-specific item semantics.

Business Constraints

Standard LLMs ignore nuanced business requirements, such as content mixing logic across movies, shows, and games.

From raw logs to a catalog ranking

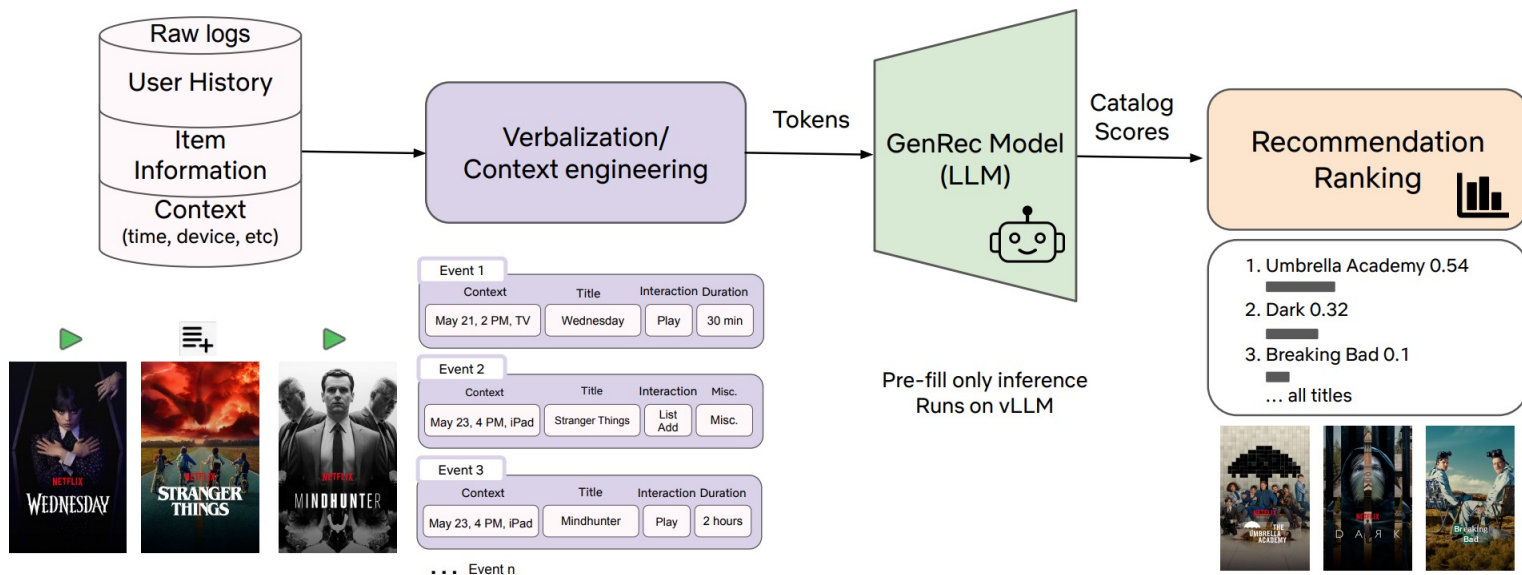


Figure 1: GenRec inference pipeline. Raw logs of user history, item metadata, and context are transformed via context engineering into natural-language prompts and fed into the GenRec model, which runs on vLLM in prefill-only mode and outputs scores for the entire item catalog, inducing a recommendation ranking.¹

The Ranking Target

$$\pi : C \rightarrow \{1, \dots, |C|\}$$

A recommender maps a request (user, context, time, history) to a ranking π of the catalog C , where $\pi(i)$ denotes the position assigned to item i .

Short-Term Engagement?

The objective is not to maximize immediate clicks or views, which can lead to clickbait or repetitive behavior.

Target: Long-Term Member Utility

The system aims to select a ranking that maximizes expected long-term member utility—a proxy for member satisfaction and retention.

A two-phase training framework

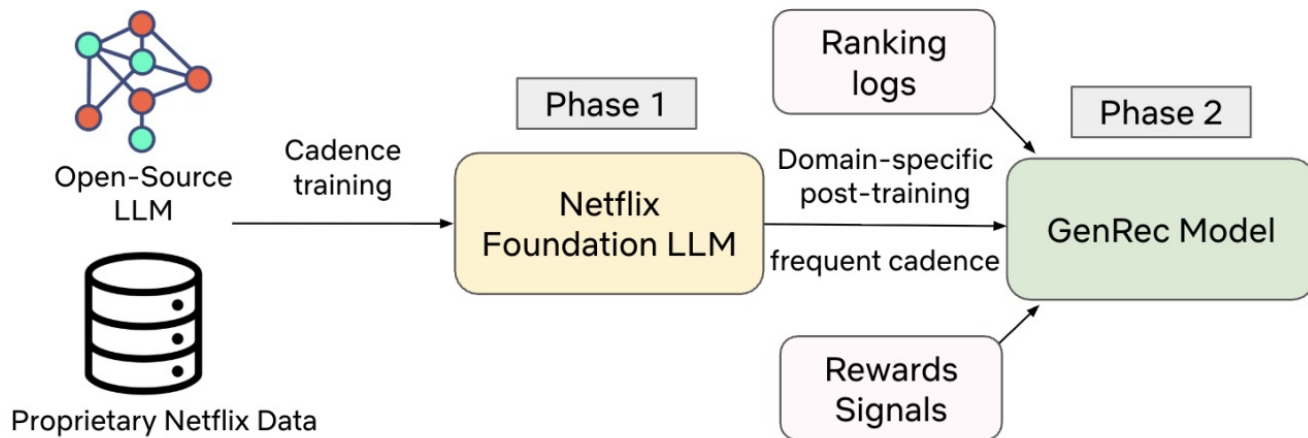


Figure 2: Two-phase training framework. Phase 1 trains a foundational LLM on Netflix data for user and content understanding, and Phase 2 post-trains on recommendation-ranking-specific data and objectives.

A two-phase training framework

	Phase I	Phase II Post-training
Capability focus	Foundational capabilities: world knowledge, user/content understanding, and language capability	recommendation ranking capabilities, such as ranking quality and recommendation steering
Update cadence	Updated relatively infrequently. It targets long-term user and content understanding.	updated more frequently to track new content launches, evolving popularity patterns, and members' latest interests
Cost sensitivity	build the most capable foundational model and is less constrained by serving cost	explicitly cost-efficient to serve largescale traffic

Model Architecture

A **decoder-only Transformer** trained with next-token-prediction style objectives. It is augmented with a **catalog-aware ranking heads** that constrains recommendations to the Netflix in-catalog items and supports efficient scoring over large candidate sets.

- (1) **Verbalization.** A verbalizer V maps the interaction history, the context, and the item metadata into a single text sequence:

$$x = V(H, \{M_i\}_{i \in C}, \tau) \quad (1)$$

- (2) **Pooled representation.** The LLM encodes x into a d -dimension representation h , taken as the hidden state at a pooling position, which summarizes the user preference and context.
- (3) **Catalog-aware scoring.** A scoring head ϕ assigns each item i a score by combining h with a learned d -dimensional embedding e_i as the recommendation ranking task.

LLM weights, scoring head, and item embeddings are trained jointly, but it does **not** disclose the initialization procedure.

Verbalization

GenRec verbalizes rich user interaction histories and context as natural language or lightly structured text.

User Message

(Input)

- Context: surface, time, device, locale, etc.
- User profile & history: country, tenure, plan, historical interactions, etc.
- Item-level detail: item IDs, item metadata (title name, release time, synopsis, etc.), popularity trends, etc.
- Task: for example, predict the next items that the user will play or thumb up

Assistant Message

(Ground Truth)

The user's implicit or explicit feedback (play, play duration, abandon, thumb, etc.), which acts as the ground-truth signal during training.

Context engineering: Context Selection and Compression

Assume we have a user with 10,000 historical interactions; you can put everything into the prompt, but it would be expensive, slow, noisy, and potentially harmful to the quality of the ranking.

- **Retain in full:** high-signal engagements, such as long-duration plays and thumbs-up events, are verbalized with richer metadata.
- **Omit entirely:** recent, low-signal events—such as very short plays or noisy views/clicks—are omitted, as they contribute little to ranking quality relative to their token cost.
- **Summarize or compress:** repetitive behaviors, such as binge-watching sessions, are compressed instead of listing every event repeatedly.
- **Elaborate selectively:** for important items (e.g. new releases or cold-start items), we can include more detailed metadata to compensate for the limited information available from the pretrained model and historical interactions.

Post-training objective

$$\begin{aligned}\mathcal{L} &= \alpha \cdot \mathcal{L}_{\text{ranking}} + \beta \cdot \mathcal{L}_{\text{language}} + \gamma \cdot \mathcal{L}_{\text{miscellaneous}} \\ \text{s.t. } &\alpha + \beta + \gamma = 1, \quad \alpha, \beta, \gamma \geq 0\end{aligned}$$

The α, β, γ are tunable hyperparameters which can be tuned through offline experiments.

Recommendation ranking objective: assign higher scores to items that correspond to high-value engagements.

Language modeling objective: over the verbalized inputs and outputs (such as predicted titles and other textual fields), to maintain the backbone's general language understanding and text generation capabilities

Recommendation ranking: Two reward categories

- **Long-term satisfaction proxies.** These rewards estimate how strongly a short-term engagement event correlates with long-term outcomes. The reward models assign higher values to engagements that **encourage members to return** to the service, **explore across a wider portion** of the catalog, or lead to **sustained engagement** over time.
- **Behavior rebalancing across content and engagement types.** Additional rewards are used to **rebalance behavior** across content types (e.g., movies vs. games vs. live vs. podcast) and **launch stage** (e.g. pre-launch, newly launched, ever-green), in order to meet business objectives such as exposure fairness across content types.

Reward-weighted ranking loss

$$\mathcal{L}_{\text{RWR}} = -\frac{1}{N} \sum_{n=1}^N w_n \log p_{\theta}(i_n^+ | x_n)$$

where the catalog probability is:

$$p_{\theta}(i_n^+ | x_n) = \frac{\exp(s_{\theta}(x_n, i_n^+))}{\sum_{j \in \mathcal{C}} \exp(s_{\theta}(x_n, j))}$$

$$w_n = F(r_n^{\text{long-term satisfaction}}, r_n^{\text{behavior rebalancing}}, \dots)$$

where each $r_n^{(k)}$ is an output from a reward model or reward signal, and F combines them into the scalar weight w_n .

x_n	Verbalized context for training example n : user history, device, time, item metadata, task, etc.
i_n^+	The positive item associated with a high-value member engagement.
$\mathcal{C}$	Catalog, or a sampled candidate set when full-catalog scoring is impractical.
$s_{\theta}(x_n, j)$	GenRec score for catalog item j , produced by the LLM representation and catalog-aware scoring head.
$p_{\theta}(i_n^+ x_n)$	Softmax probability assigned to the observed positive item.
w_n	Reward-derived scalar weight for the example.

Reward-weighted ranking loss

$$\mathcal{L}_{\text{RWR}} = -\frac{1}{N} \sum_{n=1}^N w_n \log p_{\theta}(i_n^+ | x_n)$$

- This reward-weighted loss provides a simple, robust, and cost-effective way to steer the model toward long-term value and business-aligned behavior.
- Compared to full reinforcement learning approaches, this approach is easier to deploy and maintain.
- In preliminary experiments, RL-style methods (e.g. GRPO) showed additional gains over supervised finetuning, but with a high training overhead.
- Therefore, currently we adopt the reward-weighted loss approach as the main alignment mechanism for GenRec, due to its simplicity, stability, and cost considerations.

Online Serving and Cost Optimizations

- GenRec is served with Netflix's internal LLM serving stack using the **vLLM** on NVIDIA Triton.
- **Explore smaller and distilled language models** trained on larger or more targeted datasets, aiming to recover much of the quality of larger models while reducing per-request computation
- **Optimize context length** via verbalization compaction, reducing the tokens used to represent input signals without significantly degrading offline ranking quality
- Deploy GenRec in a **prefill-only configuration**: the model consumes the input context once, and produces ranks for the full candidate set in a single forward pass.

Results: GenRec v.s. Production baseline

A large-scale A/B test allocating approximately 10% of Netflix traffic for 4 weeks.

325 million paid subscribers globally at the end of 2025

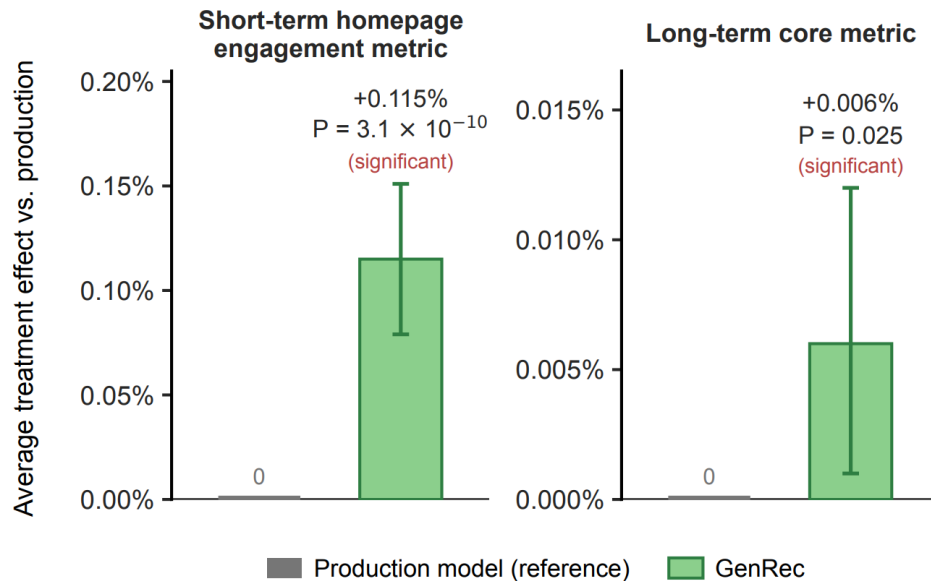


Figure 3: Online metrics of GenRec vs. production model. GenRec achieves statistically significant improvements on both short-term and long-term online metrics.²

Results: Quality Scales with Model/Data Scaling

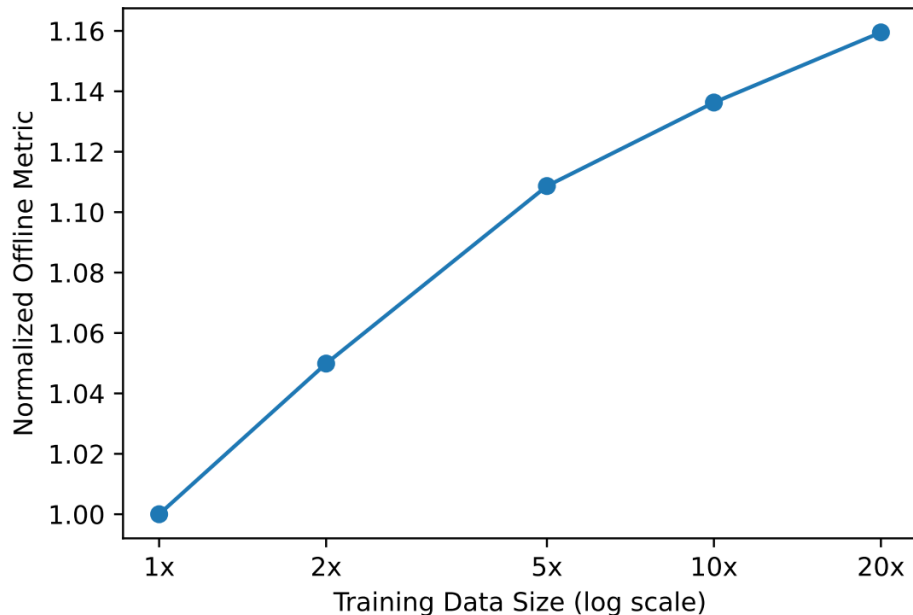


Figure 4: Phase-2 data scaling for the ~10B model.

Quality-Cost Tradeoff in Phase 2

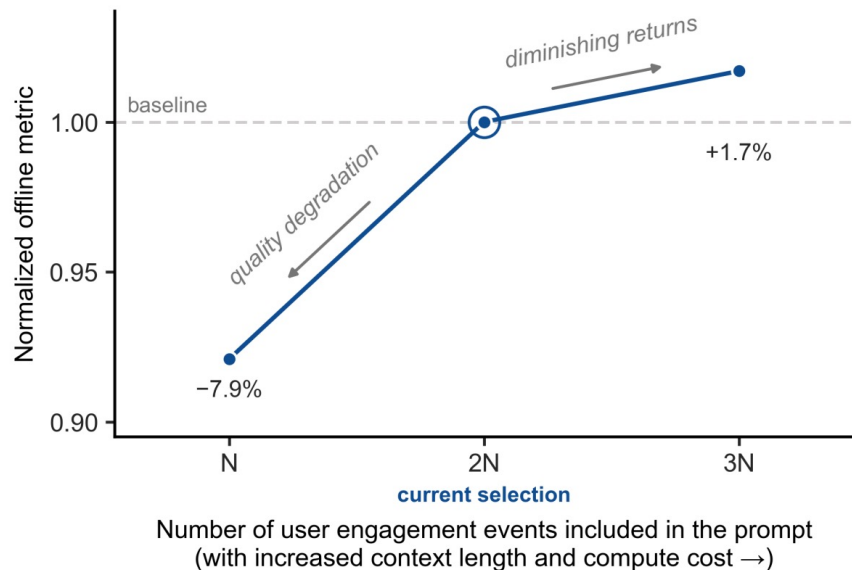
- While data and model scaling both improve quality, they also increase training and serving costs, especially trained at a higher cadence and directly serves large-scale online traffic.
- We use the data and model-scaling curves, together with context-length ablations, to identify a “sweet spot” on the **quality–cost Pareto frontier** that recovers most of the achievable quality improvement while staying within a given compute budget.

Results: Impact of Phase 1 & Phase 2 on Recommendation Task

Table 1: Impact of Phase 1 vs. Phase 2 training

	Offline ranking metrics (MRR)	Mean Reciprocal Rank
Impact of Phase 1	+ 10-20% compared to using an OSS model as base	
Impact of Phase 2	+ 35-50% compared to a freshly trained Phase 1, with the gain increasing over time	

Results: Context optimization



Quality Degradation

Truncating the history too aggressively (e.g., to N events) drops MRR by nearly 8% relative to the baseline.

Diminishing Returns

Expanding the context beyond the elbow point (2N events) yields only marginal gains (+1.7% at 3N) while linearly increasing serving cost.

Figure 5: Normalized offline ranking metric (MRR) vs. number of user engagement events included in the prompt. The dashed line marks the baseline elbow point: increasing the number of events beyond this yields diminishing returns.

LLM-Native Recommendation

From Feature Engineering to Context Engineering: LLM-centric systems prioritize constructing rich textual contexts from raw interaction sequences, content metadata, context, tool outputs, or user memory.

From Customized Architectures to Foundation Backbones: LLM-centric systems standardize on transformer backbones inherited from a foundation model. Innovation moves up a level: from per-task architecture design to questions of data, scaling laws, post-training strategy, and inference optimization

LLM-Native Recommendation

Scaling Laws as a Design Guide: Our studies show clear scaling trends in both data and model size: performance improves monotonically as we add more Phase-2 training data, and larger models consistently outperform smaller ones under fixed training budgets.

Pre-Training, Post-Training, and Reward Alignment: multiple use cases can reuse the same foundation and infrastructure, rather than re-implementing similar capabilities in separate, siloed systems.

From RecSys Infrastructure to LLM Infrastructure: the online serving stack for recommendation increasingly resembles modern LLM infrastructure: GPU-accelerated, vLLM-based, with careful batching and caching; rather than classic RecSys infrastructure built around MLPs or factorization models.

What GenRec establishes, and what it does not

It proves that an LLM-backed ranker is a viable, scalable alternative to traditional models for large-scale personalization, provided cost and infrastructure complexity are rigorously managed.

It does not eliminate the need for domain-specific adaptation, reward alignment, or strict evaluation against strong non-LLM baselines.